

SHEET DLP

```
package com.paracamplus.ilp4.exam2020.interpreter;

import com.paracamplus.ilp1.interfaces.IASTvariable;

import com.paracamplus.ilp1.interpreter.interfaces.EvaluationException;

import com.paracamplus.ilp1.interpreter.interfaces.IGlobalVariableEnvironment;

import com.paracamplus.ilp1.interpreter.interfaces.ILexicalEnvironment;

import com.paracamplus.ilp1.interpreter.interfaces.IOperatorEnvironment;

import com.paracamplus.ilp4.exam2020.interfaces.IASTfilter;

import com.paracamplus.ilp4.exam2020.interfaces.IASTsum;

import com.paracamplus.ilp4.exam2020.interfaces.IASTvisitor;

import com.paracamplus.ilp4.exam2020.interpreter.interfaces.ISum;

import com.paracamplus.ilp4.interpreter.interfaces.IClassEnvironment;

public class Interpreter extends com.paracamplus.ilp4.interpreter.Interpreter
implements IASTvisitor<Object, ILexicalEnvironment, EvaluationException> {

    public Interpreter(IGlobalVariableEnvironment globalVariableEnvironment, IOperatorEnvironment operatorEnvironment,
        IClassEnvironment classEnvironment) {

        super(globalVariableEnvironment, operatorEnvironment, classEnvironment);

    }

    @Override

    public Object visit(IASTsum iast, ILexicalEnvironment data) throws EvaluationException {

        Object[] values = new Object[iast.getValues().length];

        for (int i = 0; i < values.length; i++) {

            values[i] = iast.getValues()[i].accept(this, data);

        }

    }

}
```

```
}
```

```
return new Sum(iast.getDiscrim(), values);
```

```
}
```

```
@Override
```

```
public Object visit(IASFilter iast, ILexicalEnvironment data) throws EvaluationException {
```

```
// On vérifie que c'est bien une somme et qu'elle a le bon tag
```

```
Object maybeSum = iast.getSum().accept(this, data);
```

```
if(maybeSum instanceof ISum && ((ISum) maybeSum).getDiscrim().equals(iast.getTag())) {
```

```
ISum sum = (ISum) maybeSum;
```

```
// On vérifie que le nombre de variable est bien inférieur ou égal au nombre de valeurs
```

```
if(iast.getVars().length > sum.getValues().length) {
```

```
throw new EvaluationException("Too much variable to to bind");
```

```
ILexicalEnvironment newEnv = data;
```

```
for(int i = 0; i < iast.getVars().length; i++) {
```

```
IASVariable var = iast.getVars()[i];
```

```
newEnv = newEnv.extend(var, sum.getValues()[i]);
```

```
}
```

```
// On exécute la conséquence avec le nouvel env lex
```

```
return iast.getConsequence().accept(this, newEnv);
```

```
} else {
```

```
// On exécute l'alternant sans changer l'environnement lexical
```

```
return iast.getAlternant().accept(this, data);
```

```
}
```

```
}
```

}

Question 5 - 2.2 : Interprète [barème : 4.00 point(s)]

La correction de cette question a tenu compte des fichiers suivants :

- [eclipse-workspace/ILP4/Java/src/com/paracampus/ilp4/exam2020/interpreter/Interpreter.java](#)

Construction : appels récursifs à accept [barème : 10%, 0.40 point(s)]

Votre réponse est correcte.

0.40 / 0.40 point(s) (100%)

Construction : création de l'objet somme [barème : 15%, 0.60 point(s)]

Votre réponse est correcte.

0.60 / 0.60 point(s) (100%)

Filtre : appels récursifs à accept [barème : 10%, 0.40 point(s)]

Votre réponse est correcte.

0.40 / 0.40 point(s) (100%)

Filtre : vérification du type et du discriminant [barème : 15%, 0.60 point(s)]

Votre réponse est correcte.

0.60 / 0.60 point(s) (100%)

Filtre : vérification du nombre d'arguments et exception [barème : 15%, 0.60 point(s)]

Votre réponse est correcte.

0.60 / 0.60 point(s) (100%)

Filtre : création du nouvel environnement lexical [barème : 15%, 0.60 point(s)]

Votre réponse est correcte.

0.60 / 0.60 point(s) (100%)

Filtre : exécution de la branche then [barème : 10%, 0.40 point(s)]

Votre réponse est correcte.

0.40 / 0.40 point(s) (100%)

Filtre : exécution de la branche else [barème : 10%, 0.40 point(s)]

Votre réponse est correcte.

0.40 / 0.40 point(s) (100%)

Total pour la question : 4.00 / 4.00

Pour ajouter les sommes au compilateur on aurait besoin des classes :

`package` ilp4.exam2020.compiler; Compiler extends ilp4.compiler.Compiler implements ilp4.exam2020.interfaces.IASTvisitor

`package` ilp4.exam2020.compiler; FreeVariableCollector extends ilp4.compiler.FreeVariableCollector implements ilp4.exam2020.interfaces.IASTvisitor

`package` ilp4.exam2020.compiler; GlobalVariableCollector extends ilp4.compiler.GlobalVariableCollector implements ilp4.exam2020.interfaces.IASTvisitor

`package` ilp4.exam2020.compiler.normalizer; Normalizer extends ilp4.compiler.normalizer.Normalizer implements ilp4.exam2020.interfaces.IASTvisitor

`package` ilp4.exam2020.compiler.normalizer; INormalizationFactory extends ilp4.compiler.normalizer.INormalizationFactory

`package` ilp4.exam2020.compiler.normalizer; NormalizationFactory extends ilp4.compiler.normalizer.NormalizationFactory implements ilp4.exam2020.compiler.normalizer.INormalizationFactory

`package` ilp4.exam2020.compiler.interfaces; IASTCsum extends ilp4.exam2020.interfaces.IASTsum

`package` ilp4.exam2020.compiler.interfaces; IASTCfilter extends ilp4.exam2020.interfaces.IASTfilter

`package` ilp4.exam2020.compiler.ast; ASTCsum extends ilp4.exam2020.ast.ASTsum implements ilp4.exam2020.compiler.interfaces.IASTCsum

`package` ilp4.exam2020.compiler.ast; ASTCfilter extends ilp4.exam2020.ast.ASTfilter implements ilp4.exam2020.compiler.interfaces.IASTCfilter

`package` ilp4.exam2020.compiler.test; CompilerTest extends ilp4.compiler.test.CompilerTest

`package` com.paracamplus.ilp4.exam2020.compiler;

`import` java.util.Set;

`import` com.paracamplus.ilp1.compiler.CompilationException;

`import` com.paracamplus.ilp1.compiler.interfaces.IASTCglobalVariable;

`import` com.paracamplus.ilp4.exam2020.interfaces.IASTfilter;

```

import com.paracamplus.ilp4.exam2020.interfaces.IASTsum;

import com.paracamplus.ilp4.exam2020.interfaces.IASTvisitor;

public class GlobalVariableCollector

extends com.paracamplus.ilp4.compiler.GlobalVariableCollector

implements IASTvisitor<Set<IASTCglobalVariable>, Set<IASTCglobalVariable>, CompilationExcepti
on> {

    @Override

    public Set<IASTCglobalVariable> visit(IASTfilter iast, Set<IASTCglobalVariable> data) throws Com
pilationException {

        data = iast.getSum().accept(this, data);

        data = iast.getConsequence().accept(this, data);

        data = iast.getAlternant().accept(this, data);

        return data;

    }

    @Override

    public Set<IASTCglobalVariable> visit(IASTsum iast, Set<IASTCglobalVariable> data) throws Com
pilationException {

        return null;

    }

}

package com.paracamplus.ilp4.exam2020.compiler;

import java.util.Arrays;

import java.util.HashSet;

import java.util.Set;

import com.paracamplus.ilp1.compiler.CompilationException;

import com.paracamplus.ilp1.compiler.interfaces.IASTClocalVariable;

```

```

import com.paracamplus.ilp1.interfaces.IASTvariable;

import com.paracamplus.ilp2.compiler.interfaces.IASTCfunctionDefinition;

import com.paracamplus.ilp4.compiler.interfaces.IASTCprogram;

import com.paracamplus.ilp4.exam2020.interfaces.IASTfilter;

import com.paracamplus.ilp4.exam2020.interfaces.IASTsum;

import com.paracamplus.ilp4.exam2020.interfaces.IASTvisitor;

public class FreeVariableCollector

extends com.paracamplus.ilp4.compiler.FreeVariableCollector

implements IASTvisitor<Void, Set<IASTClocalVariable>, CompilationException>{

    public FreeVariableCollector(IASTCprogram program) {

        super(program);

    }

    @Override

    public Void visit(IASTfilter iast, Set<IASTClocalVariable> data) throws CompilationException {

        Set<IASTClocalVariable> newvars = new HashSet<>();

        iast.getConsequence().accept(this, newvars);

        IASTvariable[] vars = iast.getVars();

        newvars.removeAll(Arrays.asList(vars));

        try {

            IASTCfilter filter = (IASTCfilter) iast;

            filter.setConsequenceVars(newvars);

            for (IASTvariable v : newvars) {

```

```
        ((IASTClocalVariable) v).setClosed();
    }
} catch (ClassCastException exc) {
    throw new RuntimeException("should not occur");
}

data.addAll(newvars);

return null;
}

@Override

public Void visit(IASTsum iast, Set<IASTClocalVariable> data) throws CompilationException {

    return null;
}
}
```